

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Информационный поиск»

Студент: А. М. Голев
Преподаватель: А. А. Кухтичев
Группа: М8О-401Б
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №2 «Поисковый робот»

Необходимо написать поисковый робот — компонент обкачки документов, используя любой язык программирования.

- Единственным аргументом поисковому роботу подаётся путь до yaml-конфига, содержащий:
 - данные для базы данных в секции `db`;
 - данные для робота в секции `logic`: задержка между обкачкой страницы;
 - любые другие данные, необходимые для реализации логики поискового робота.
- Сохранять в базе данных (например, MongoDB) документы со следующими полями:
 - `url`, нормализованный;
 - «сырой» html-текст документа;
 - название источника;
 - дата обкачки документа в формате Unix time stamp.
- Поисковый робот можно остановить в любой момент, и при повторном запуске робот должен начать с того документа, с которого он остановился.
- Периодически он должен уметь переобкачивать документы, которые уже есть в базе, но только в том случае, если они изменились.

1 Описание

1 Общее устройство

Поисковый робот — программа `robot.py` на Python 3 (библиотеки `requests`, `pymongo`, `PyYAML`). Вся логика — один класс `MusicCrawler`. Робот обкачивает те же два источника, что использовались при сборке корпуса в первой лабораторной работе: `Lyrics.ovh` API и `MusicBrainz` API. Хранилище — `MongoDB`.

Робот не бродит по ссылкам: список URL для обкачки строится из метаданных корпуса из первой лабораторной работы (файлы `*.json` в каталогах `metadata`). Для каждого трека из метаданных собирается URL вида `https://api.lyrics.ovh/v1/{артист}/{название}`, для каждой записи `MusicBrainz` — `https://musicbrainz.org/ws/2/recording/{mbid}?fmt=json`. Таким образом, корпус из первой работы выступает seed-списком, а робот переобкачивает те же документы и следит за их изменениями.

2 Конфигурация

Единственный аргумент робота — путь до YAML-конфига. В нём три обязательные секции: `db` (подключение к `MongoDB`), `logic` (задержка между обкачкой, лимит документов на источник, интервал переобкачки, таймауты, число повторных попыток) и `sources` (источники данных и пути к метаданным корпуса). Дополнительно — секции `robots_txt` и `logging`.

```
1 db:
2   host: localhost
3   port: 27017
4   database: robot
5   collection: documents
6
7 logic:
8   delay_between_requests: 2.0
9   max_documents_per_source: 15000
10  refresh_interval_days: 7
11  retry_attempts: 3
12  timeout_seconds: 30
13
14 sources:
15   lyrics_ovh:
16     metadata_dir: "../lab1/lyrics_corpus/metadata"
17     source_name: "Lyrics.ovh"
18   musicbrainz:
19     metadata_dir: "../lab1/musicbrainz_corpus/metadata"
20     source_name: "MusicBrainz"
21     respect_robots_txt: false # /ws/2 is the official public API
```

3 Схема документа в MongoDB

Каждый обкачанный документ — запись в коллекции `documents`:

Поле	Описание
<code>url</code>	нормализованный URL (scheme + host + path, для MusicBrainz — без query-параметров); уникальный индекс
<code>html_content</code>	«сырой» текст ответа (для API-источников — тело JSON-ответа как есть)
<code>source</code>	название источника (Lyrics.ovh / MusicBrainz)
<code>crawl_timestamp</code>	дата обкачки, Unix time stamp
<code>content_hash</code>	MD5-хэш содержимого, для проверки изменений
<code>title, content_length, ...</code>	заголовок документа и служебные поля (код ответа, Content-Type, Last-Modified)
<code>previous_versions</code>	история предыдущих версий изменившихся документов (хэш, дата, заголовок)

4 Возобновление работы

После каждого успешно сохранённого документа робот записывает контрольную точку — последний обработанный URL — в отдельную коллекцию `crawler_checkpoints`. При запуске точка загружается, список URL строится заново (детерминированно, из тех же метаданных), и робот продолжает со следующего после контрольной точки документа. Остановка — по `Ctrl+C` (SIGINT/SIGTERM): робот завершает текущий документ, сохраняет статистику и выходит.

5 Переобкачка изменившихся документов

Документ переобкачивается, если с прошлой обкачки прошло больше `refresh_interval_days` дней. Скачанный контент сравнивается по MD5-хэшу с сохранённым: если не изменился — обновляется только `crawl_timestamp`; если изменился — документ обновляется, а старая версия попадает в `previous_versions`.

6 Особенности реализации

При написании кода пришлось учесть особенности источников:

- оба источника — JSON API, а не HTML-сайты: робот принимает ответы с Content-Type `text/html` и `application/json`, «сырой» текст ответа сохраняется как есть;

- robots.txt MusicBrainz содержит **Disallow: /ws**, под который попадает официальное публичное API **/ws/2**; поэтому проверка robots.txt отключается отдельно для этого источника (параметр **respect_robots_txt**), для остальных источников она включена;
- в метаданных корпуса есть дубли (одинаковые пары артист—название), поэтому список URL дедуплицируется перед обкачкой;
- сетевые сбои: повторные попытки запроса (3 раза, с backoff), таймаут 30 с, пауза 2 с между запросами, чтобы не долбить API.

7 Статистика запуска

Статистика приведена для целевого размера корпуса — 30000 документов (по 15000 из каждого источника); значения экстраполированы по контрольному прогону (108 документов за 292 с, прерывание по **Ctrl+C**).

Показатель	Значение
Количество документов	30000
Скорость обкачки	~0,37 док/с (задержка 2 с + время запроса)
Время полной обкачки	~22–23 часа
Средний размер сырого ответа	~1 КБ (тексты песен ~1,5 КБ, записи ~0,3 КБ)
Объём сырых данных в БД	~30 МБ
Ошибки обкачки	~0% (в контрольном прогоне — 0 из 108)
Интервал переобкачки	7 дней

Возобновление работы проверено: после остановки и повторного запуска робот загрузил контрольную точку и продолжил со следующего документа (**Продолжаем с позиции 2** в логе).

2 Исходный код

Программа `robot.py` состоит из класса `MusicCrawler`. Основные блоки: построение списка URL из метаданных, обкачка одного URL (`crawl_url`), сохранение документа с проверкой изменений (`process_document`), контрольные точки (`save_checkpoint/load_checkpoint`) и основной цикл (`run`).

Обкачка одного URL: нормализация, проверка `robots.txt`, запрос с таймаутом, проверка Content-Type, вычисление хэша:

```
1 def crawl_url(self, url, source_name, respect_robots_txt=True):
2     normalized_url = self.normalize_url(url, source_name)
3     if respect_robots_txt and not self.check_robots_txt(url, source_name):
4         return None
5     response = self.session.get(url, timeout=...)
6     response.raise_for_status()
7     content = response.text
8     content_type = response.headers.get('Content-Type', '')
9     if 'text/html' not in content_type and 'json' not in content_type:
10        return None
11    return {
12        'url': normalized_url,
13        'html_content': content,
14        'content_hash': self.calculate_content_hash(content),
15        'source': source_name,
16        'crawl_timestamp': int(time.time()),
17        ...
18    }
```

Сохранение документа: сравнение хэша с сохранённым, обновление только изменившихся, история предыдущих версий:

```
1 def process_document(self, document):
2     existing = self.collection.find_one({'url': document['url']})
3     if existing:
4         if existing.get('content_hash') == document['content_hash']:
5             # unchanged: only refresh the crawl date
6             self.collection.update_one(
7                 {'_id': existing['_id']},
8                 {'$set': {'crawl_timestamp': document['crawl_timestamp']}})
9         else:
10            # changed: old version goes to history, document updated
11            document['previous_versions'] = existing.get(
12                'previous_versions', []) + [{
13                    'content_hash': existing.get('content_hash'),
14                    'crawl_timestamp': existing.get('crawl_timestamp')}]
15            self.collection.update_one(
16                {'_id': existing['_id']}, {'$set': document})
17     else:
18        self.collection.insert_one(document)
```

Контрольная точка — последний обработанный URL (коллекция `crawler_checkpoints`, одна запись на робота):

```
1 def save_checkpoint(self, url):
2     self.db['crawler_checkpoints'].update_one(
3         {'crawler_id': 'music_crawler'},
4         {'$set': {'last_url': url, 'timestamp': time.time()}},
5         upsert=True)
```

Основной цикл: пропуск ещё свежих документов, обкатка, задержка между запросами:

```
1 for i in range(start_index, len(all_urls)):
2     if not self.running: # stop on Ctrl+C
3         break
4     url_info = all_urls[i]
5     existing = self.collection.find_one({'url': url_info['url']})
6     if existing and not self.should_recrawl(existing):
7         continue # too early to re-crawl
8     document = self.crawl_url(url_info['url'], ...)
9     if document:
10         self.process_document(document)
11         self.save_checkpoint(url_info['url'])
12     time.sleep(self.config['logic']['delay_between_requests'])
```

3 Выводы

Выполнив вторую лабораторную работу по курсу «Информационный поиск», я написал поискового робота — компонент обкачки документов. Основное, что я узнал нового:

- устройство краулера на практике: seed-список URL, нормализация URL, вежливые задержки между запросами, обработка robots.txt;
- хранение документов в MongoDB: схема документа с нормализованным URL, сырым текстом, источником и датой обкачки, индексы по url и хэшу содержимого;
- возобновляемость: контрольные точки в отдельной коллекции позволяют остановить робота в любой момент и продолжить с того же места при перезапуске;
- переобкачка по изменению: сравнение MD5-хэша содержимого позволяет не трогать неизменившиеся документы и сохранять историю версий изменившихся;
- реальность снова внесла поправки: источники оказались JSON API, а не HTML-сайтами, а robots.txt MusicBrainz формально запрещает доступ к собственному публичному API — пришлось делать проверку robots.txt настраиваемой для каждого источника.

Скорость обкачки упирается не в сеть, а в задержку между запросами (2 с): полный проход по корпусу из 30000 документов занимает почти сутки. Для следующих лабораторных работ в базе теперь есть актуальные документы с датами обкачки и историей версий, на которых можно строить поисковый индекс.

Список литературы

- [1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д. А. Ключина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))
- [2] Список использованных источников оформлять нужно по ГОСТ Р 7.05-2008
- [3] MongoDB Documentation // <https://www.mongodb.com/docs/>
- [4] PyMongo 4.x Documentation // <https://pymongo.readthedocs.io>
- [5] PyYAML Documentation // <https://pyyaml.org/wiki/PyYAMLDocumentation>
- [6] A Standard for Robot Exclusion // <https://www.robotstxt.org/orig.html>
- [7] Lyrics.ovh API // <https://api.lyrics.ovh>
- [8] MusicBrainz API Documentation // https://musicbrainz.org/doc/MusicBrainz_API
- [9] Requests: HTTP for Humans // <https://docs.python-requests.org>